Session 27 Teamwork — A Real Per-Project Activity Feed

Context

Work in your team (six teams of four students). LIVE already built real comment create/list and the real inline `@mention` autocomplete. Your job: build a real, render-only Activity feed on Project Detail — fetching the real backend's own activity log, and narrowing it down to just this project's own entries, using the identical real client-side-filter shape Backlog and `SortableIssueList` already demonstrated (not inventing a new one, and never asking the backend for a per-project route that does not exist).

Goal

A real "Activity" section on Project Detail, listing this project's own recent real activity entries (newest first), each as a plain-English sentence with its own timestamp — never another project's entries, never a raw workspace-level entry with no connection to any issue.

What to build

- A real new frontend type matching the backend's own activity row shape — think about which fields a frontend consumer of this data actually needs (what happened, which issue, who did it, when).
- A real, render-only component that takes a list of entries as a prop and renders one row per entry — no fetch, no mutation inside it at all.
- A real lookup from the backend's own raw `action` value to a plain-English sentence — think about what should happen for an `action` value this lookup doesn't explicitly handle (the backend may add new ones later).
- Wiring on Project Detail: a real second fetch (the workspace's own full activity log), a real filter narrowing it to this project's own issues, and the new component rendered below the existing issue list. Think carefully about WHICH real issue list you narrow against — Project Detail's own issue fetch is affected by the Issues UI's own status/title filter and search, but this project's own activity membership must never change just because a teammate is currently filtering the Issues list.

Rules

Allowed: everything already shown LIVE this session (`apiFetch` with a real token, the generic render-only list-component shape), plus the real backend's own Activity route (already built before this session — read its real shape, don't guess it).

Not yet: no Notifications UI, no comment deletion, no Dashboard page — all of that is this session's own homework, not teamwork.

Test it

inside api/

BASH

```
PORT=3000 npm run start
```

inside web/ (second terminal)

BASH

```
npm run dev
```

in a third terminal

BASH

```
npx tsc --noEmit
```

Then, in a real browser: open a project's own detail page, post a comment with a real `@mention` on one of its issues (from LIVE), and confirm a real new Activity entry for that action appears on Project Detail — but NOT on a different project's own detail page. Then apply a real status/title filter in the Issues UI that hides that same issue from the list, and confirm its own Activity entry is STILL shown — the Activity section must never react to the Issues UI's own filter.

Done when

- The real Activity section renders on Project Detail, below the existing issue list.
- It lists only this project's own real recent entries, newest first.
- Applying a status/title filter to the Issues list never removes an entry from the Activity section — the two are genuinely independent.
- A workspace-level entry with no connection to any issue is correctly excluded.
- `npx tsc --noEmit` exits clean.

Working time

About 16 minutes total (TW-01..04) — this session's own TEAMWORK task is deliberately small; the one hard new mechanism this session teaches (the `@mention` picker) is already built by LIVE.